



GESTIAPRO: GESTIÓN DE PROYECTOS COLABORATIVOS

DOCUMENTACIÓN TÉCNICA Y GUÍA DE USUARIO

AUTORES: CONTRERAS CERRON DAVID ANTHONY 100% – 74465364@CONTINENTAL.EDU.PE
GARCIA BETANCOURT JOSUE DANIEL 100% – 77043114@CONTINENTAL.EDU.PE
HUAMAN QUIÑONES ANGHELO JOSUE 100% – 73545882@CONTINENTAL.EDU.PE
PAYANO SALVADOR CESAR ALEJANDRO 100% – 70231150@CONTINENTAL.EDU.PE
TORRE MEDINA RAUL ALEJANDRO 99% – 77331712@CONTINENTAL.EDU.PE
URBANO FABIAN JORGE JOSE – 71700320@CONTINENTAL.EDU.PE
VILCAPOMA CAMPOSANO YONY – 71197528@CONTINENTAL.EDU.PE

Huancayo - Perú 2025

ÍNDICE

Resumen ejecutivo.....	3
Análisis de requerimientos.....	4
Diseño del sistema.....	6
Arquitectura de Software.....	6
Diagrama de Clases.....	7
Despliegue.....	9
Guía de Usuario.....	11
1.1. Inicio de Sesión.....	11
1.2. Crear y Gestionar Proyectos.....	11
1.3. Gestionar Épicas.....	11
1.4. Chat en Tiempo Real.....	12
1.5. Panel Administrativo.....	12
1.6. Integración con IA.....	13
1.7. Documentación Adicional.....	14
1.8. Informe Final.....	14
1.9. Licencia y Créditos.....	15
1.10. Soporte y Contacto.....	15
Conclusiones y recomendaciones.....	16

RESUMEN EJECUTIVO

El presente proyecto consiste en el diseño e implementación de una plataforma web llamada GestIAPro, orientada a la gestión colaborativa de proyectos con apoyo de herramientas de Inteligencia Artificial. La propuesta responde a la necesidad de mejorar la planificación, control y seguimiento de proyectos en equipos multidisciplinares, aprovechando tecnologías modernas como el stack MERN (MongoDB, Express.js, React.js y Node.js).

En cuanto a los requerimientos del sistema, se definieron tanto requisitos funcionales como no funcionales. Los funcionales abarcan la autenticación de usuarios, gestión de proyectos y tareas, interacción en tiempo real mediante chat, y la generación de historias de usuario mediante IA. Los requisitos no funcionales se centran en la escalabilidad, seguridad, facilidad de uso y disponibilidad. Esta estructura permitió establecer una base sólida para el desarrollo iterativo del sistema.

El diseño de la base de datos fue modelado utilizando MongoDB, lo que ofreció flexibilidad para representar entidades como usuarios, proyectos, tareas, mensajes y métricas de desempeño. La estructura se pensó en función de relaciones documentales con alta eficiencia en consultas y escalabilidad. Se elaboró un diagrama entidad-relación (E/R) para visualizar claramente las colecciones, sus atributos y vínculos.

Asimismo, se desarrolló un diagrama de clases para modelar la lógica de negocio, incluyendo clases como Usuario, Proyecto, Tarea, Mensaje e HistoriaUsuario, con sus respectivos atributos y métodos. Este diagrama guió la implementación del backend bajo principios de orientación a objetos, favoreciendo la claridad estructural y la reutilización de código.

La arquitectura del software sigue el patrón cliente-servidor, en donde el frontend (React.js) se comunica mediante una API RESTful con el backend (Node.js + Express). Esta arquitectura se dividió en capas como controladores, rutas, modelos y servicios de IA, lo que permitió una separación de responsabilidades y una mayor mantenibilidad. El sistema también permite traducción automática y generación inteligente de historias de usuario como parte de su integración con IA.

Finalmente, el despliegue del sistema se realizó a través de la plataforma Railway, sin el uso de contenedores Docker. El backend y frontend fueron conectados directamente desde sus respectivos repositorios en GitHub, y se configuró MongoDB como servicio persistente. Para la conexión entre servicios, se emplearon variables de entorno (.env), y Railway permitió gestionar rutas de despliegue, logs y control de errores desde una interfaz unificada. Aunque actualmente el despliegue ha fallado, la infraestructura está preparada para su correcta puesta en producción una vez resueltos los errores.

En conjunto, GestIAPro demuestra cómo una solución moderna, diseñada con fundamentos sólidos en arquitectura y despliegue, puede ofrecer una experiencia robusta para la gestión de proyectos en entornos colaborativos potenciados por inteligencia artificial.

ANÁLISIS DE REQUERIMIENTOS

Requerimientos Funcionales	Casos de Uso
La aplicación permite gestionar proyectos	- Crear, leer, actualizar o eliminar proyecto. (CRUD) - Asignar historia de usuario a miembro del equipo - Visualizar tablero Kanban
La aplicación permite la comunicación en tiempo real	- Enviar mensajes en el chat - Recibir mensajes en tiempo real
El sistema permite la traducción de mensajes y la predicción de historias de usuario en base al título y a la épica del proyecto	- Traducir mensajes de otro idioma, al idioma nativo - Predecir posibles historias de usuario en base al título y la épica de los proyectos. - Editar la predicción acorde al proyecto
El sistema permite acceder al panel de administración	- Visualizar proyectos activos, historias de usuario, épicas en progreso, miembros del equipo, tareas vencidas, progreso promedio, versiones pendientes y estado de proyectos.
Requerimientos No Funcionales	
Utilizar el stack MERN (MongoDB, Express.js, React.js, Node.js) para el desarrollo de la aplicación web.	
Usar Git como sistema de control de versiones y alojar el código en un repositorio público en GitHub, siguiendo un flujo de trabajo claro (Git Flow o GitHub Flow).	
Incluir pruebas unitarias, de integración y de interfaz de usuario (E2E), utilizando herramientas como Jest, Mocha, Cypress o React Testing Library.	
Aplicar una metodología ágil (Scrum o Kanban) durante el desarrollo del proyecto, con sprints semanales y entregas incrementales.	
Diseñar una interfaz intuitiva, fácil de usar y accesible, con soporte responsive para dispositivos móviles y escritorio.	
Garantizar tiempos de respuesta rápidos y escalabilidad para soportar múltiples usuarios concurrentes.	
Asegurar un alto tiempo de disponibilidad de la aplicación durante su ejecución y demostraciones.	

DISEÑO DEL SISTEMA

ARQUITECTURA DE SOFTWARE

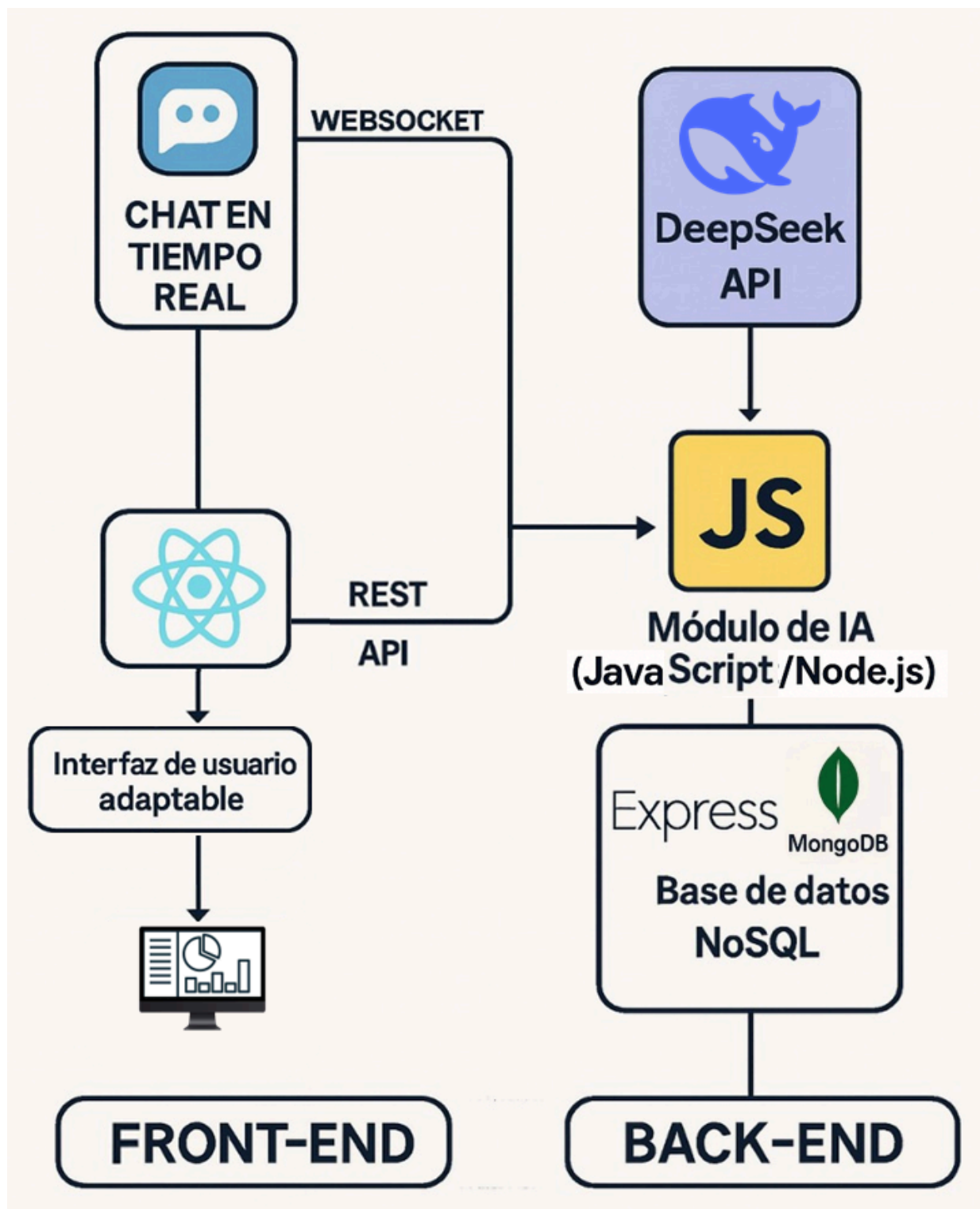
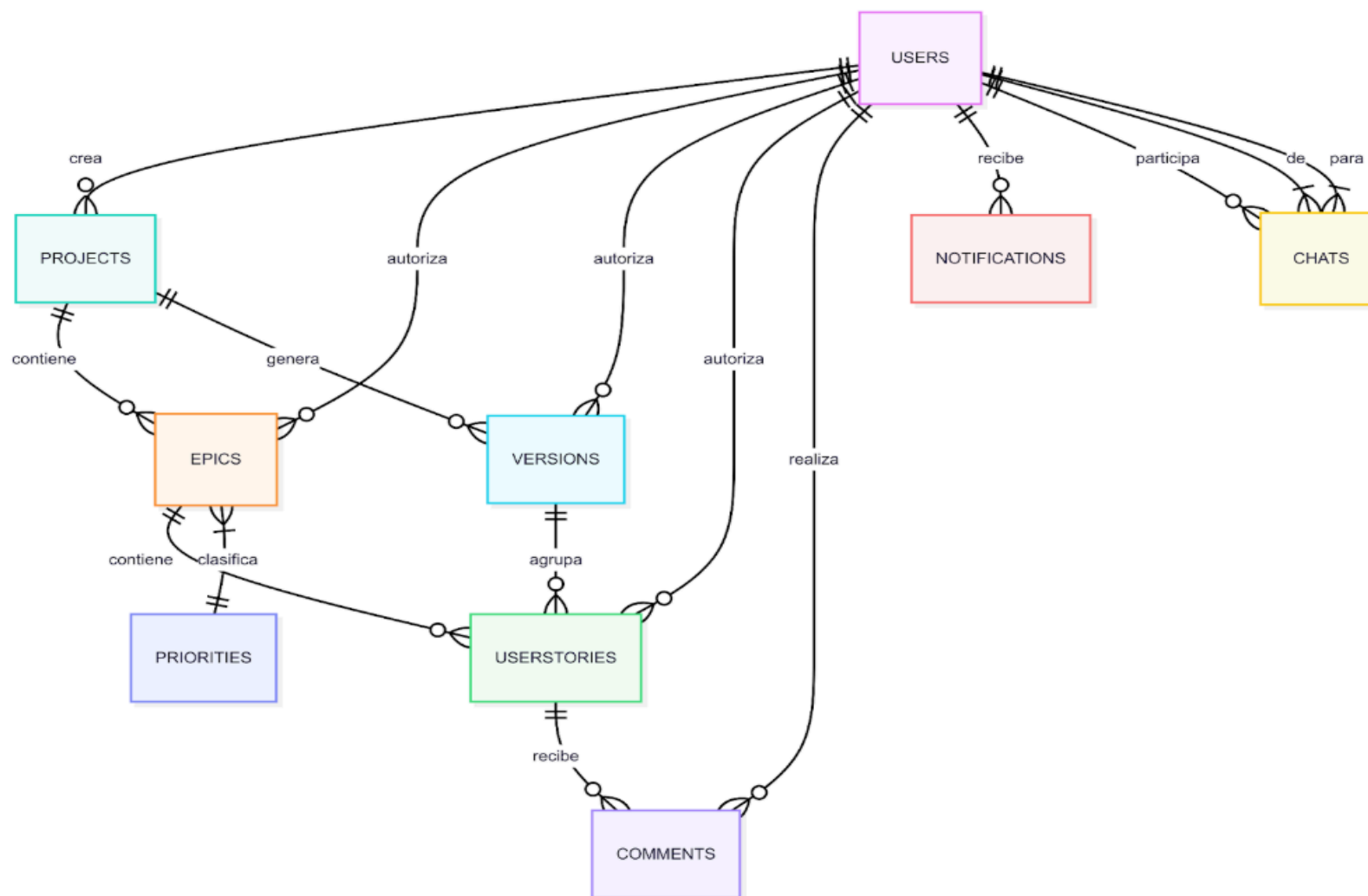


DIAGRAMA DE CLASES



MODELO LÓGICO DE BD



DESPLIEGUE

El sistema GestIAPro fue desplegado exitosamente utilizando la plataforma Railway, la cual ofrece un entorno en la nube moderno para aplicaciones del stack MERN. Esta solución permitió implementar y gestionar los tres componentes fundamentales del sistema: backend, frontend y base de datos MongoDB, todo desde una interfaz centralizada y con mínima configuración.

ARQUITECTURA DEL DESPLIEGUE

La arquitectura de producción está compuesta por:

- Frontend (React + Tailwind CSS): desplegado directamente desde el repositorio GitHub conectado a Railway. Su propósito es ofrecer una interfaz de usuario adaptable, moderna y responsive.
- Backend (Node.js + Express): también desplegado desde GitHub. Expone una API RESTful y WEBSOCKET server.
- MongoDB (NoSQL): alojado como servicio persistente en Railway, conectado mediante la librería Mongoose y configurado para mantener datos de usuarios, proyectos, tareas y métricas.

PROCESO DEL DESPLIEGUE

- Conexión con GitHub: Se enlazaron las carpetas del frontend y backend a Railway ubicadas en el mismo repositorio. Esto habilitó el despliegue automático en cada push a la rama main.
- Configuración de variables de entorno: Para cada servicio se definieron valores clave como la cadena de conexión a MongoDB, claves de acceso a la API de IA, y configuraciones del entorno (puerto, CORS, JWT, etc.).
- Implementación modular: Railway desplegó cada servicio de forma independiente, facilitando el seguimiento de errores y control de versiones por separado.
- Monitorización en tiempo real: Desde la sección de Activity, Railway ofrece trazabilidad detallada de cada evento, permitiendo verificar que los despliegues recientes fueron exitosos, como se evidencia en la imagen (redployment successful para backend, frontend y MongoDB hace solo 3 minutos).
- MongoDB vía Docker: Aunque el frontend y backend no utilizaron contenedores Docker, la base de datos MongoDB fue aprovisionada utilizando una imagen Docker interna gestionada por Railway, simplificando la persistencia y escalabilidad.

RESULTADOS DEL DESPLIEGUE

- Tiempo estimado de despliegue completo: menos de 5 minutos.
- Reintentos y actualizaciones: Railway permitió realizar redeployments manuales tras ajustes en el código o configuraciones, como se muestra en la secuencia de eventos recientes.
- Visibilidad de errores: en caso de fallos, la consola de Railway ofrecía mensajes detallados por servicio, permitiendo corrección inmediata.
- URLs asignadas automáticamente: cada componente cuenta con su propio subdominio de acceso.

The screenshot displays the GestiAPro dashboard for the 'gestionproyectoscolaborativos' project in the 'production' environment. The dashboard features a grid-based architecture view with three main components: MongoDB, backend, and frontend. The MongoDB component is connected to the backend and frontend via Docker Image and stellar-volume. The backend and frontend components are connected via GitHub. The activity log on the right shows recent deployments and redeployments for the backend, frontend, and MongoDB components.

Architecture

- MongoDB**
 - 3 minutes ago via Docker Image
 - stellar-volume
- backend** (gestionproyectoscolaborativ...)
 - 3 minutes ago via GitHub
- frontend** (gestionproyectoscolaborativ...)
 - 3 minutes ago via GitHub

Activity

- backend**
 - Redeployment successful (3 mins ago)
- frontend**
 - Redeployment successful (3 mins ago)
- backend**
 - Deployment redeployed (3 mins ago)
- MongoDB**
 - Redeployment successful (3 mins ago)
- frontend**
 - Deployment successful (4 hours ago)
- backend**

[Set up your project locally](#)

GUÍA DE USUARIO

1.1. Inicio de Sesión

Acceso al Sistema

2. **Abre tu navegador web** y accede a: `http://localhost:5173`
3. **Ingresa tus credenciales:**
 - **Correo electrónico**
 - **Contraseña**
4. **Haz clic en "Iniciar Sesión"** para acceder al dashboard principal

1.2. Crear y Gestionar Proyectos

Crear Nuevo Proyecto

1. **Navega a Proyectos:** Desde el panel lateral, selecciona "Proyectos"
2. **Crear proyecto:** Haz clic en el botón "Nuevo Proyecto"
3. **Completa la información:**
 - **Nombre del proyecto**
 - **Descripción**
 - **Fecha de inicio y fin**
 - **Miembros del equipo**
4. **Guarda el proyecto** y comienza a añadir épicas

1.3. Gestionar Épicas

Tablero Kanban

1. **Accede al tablero:** En cada proyecto, haz clic en "Tablero Kanban"
2. **Crear nueva épica** con los siguientes datos:
 - **Título**
 - **Descripción**
 - **Responsable** (selecciona un miembro del equipo)
 - **Prioridad:** Alta, Media o Baja
 - **Fecha límite**

Estados de las Épicas

Arrastra las épicas entre las siguientes columnas para actualizar su estado:

Estado	Descripción
 Pendiente	Tareas por comenzar
 En Progreso	Tareas en desarrollo
 Completado	Tareas finalizadas

1.4. Chat en Tiempo Real

Uso del Chat

1. **Accede al chat:** En la sidebar del sistema, haz clic en "Chat"
2. **Selecciona destinatario:** Elige al usuario al que deseas enviar el mensaje
3. **Envía mensajes:** Escribe tu mensaje en el campo inferior y presiona **Enter**
4. **Comunicación instantánea:** Los mensajes aparecerán en tiempo real para emisor y receptor

1.5. Panel Administrativo

Métricas y Estadísticas

El panel administrativo te permite visualizar:

- **Proyectos Activos**
- **Historias de Usuario**
- **Épicas en Progreso**
- **Miembros del Equipo**
- **Tareas Vencidas**
- **Progreso Promedio**
- **Versiones Pendientes**
- **Estado de Proyectos**

1.6. Integración con IA

Traducción Automática en el Chat

Funcionalidad:

- **Detección automática** del idioma del mensaje
- **Traducción en tiempo real** para el receptor
- **Comunicación efectiva** entre usuarios de diferentes idiomas

Ejemplo de uso:

 Usuario 1 escribe: "Meeting tomorrow at 10 AM"

 Usuario 2 ve: "Reunión mañana a las 10 AM"

Generación de Historias de Usuario

Proceso:

1. **Crear historia:** Al crear una nueva historia de usuario, haz clic en "Generar historia de usuario"
2. **Análisis automático:** El sistema analiza el título y descripción
3. **Generación ágil:** Produce una historia con formato ágil estándar

Ejemplo:

Título: "Implementar inicio de sesión"

Épica: "Autenticación"

Salida generada:

"Como usuario, quiero poder iniciar sesión con mi correo para acceder al sistema de forma segura."

1.7. Documentación Adicional

README.md

El archivo README.md incluye:

- Descripción general del sistema
- Características destacadas
- Instrucciones de instalación y ejecución
- Tecnologías utilizadas
- Créditos y licencia

Diagramas de Flujo y Arquitectura

Ubicación: Carpeta /docs del repositorio

Herramientas sugeridas:

- Lucidchart
- Draw.io
- Figma
- PlantUML

1.8. Informe Final

Decisiones de Diseño

- **Arquitectura cliente-servidor** para escalabilidad
- **Separación de responsabilidades** entre frontend y backend
- **Comunicación en tiempo real** con Socket.IO
- **Integración de IA** para asistente virtual y predicción de tiempos

Desafíos Enfrentados

- Sincronización en tiempo real entre múltiples usuarios
- Manejo de tokens JWT para seguridad
- Entrenamiento de modelos de IA con datos limitados

Soluciones Implementadas

- **Redis** para compartir estados entre servidores
- **Middlewares de autenticación** seguros
- **Pruebas unitarias y de integración** automatizadas
- **Docker** para entornos consistentes

1.9. Licencia y Créditos

Licencia

Este proyecto está bajo la **licencia MIT**

Equipo de Desarrollo

- **Josué Daniel García Betancourt**
- **Contreras Cerron David**
- **Torre Medina Raul**
- **Huamán Quiñones Anghelo Josue**
- **Vilcapoma Camposano Yony**
- **Urbano Fabian Jorge Jose**
- **Payano Salvado Cesar Alejandro**

Institución Académica

- **Universidad Continental**
- **Curso:** Taller de Proyectos 2 – Ingeniería de Sistemas e Informática

1.10. Soporte y Contacto

Obtener Ayuda

Para reportar errores o solicitudes de mejora:

- **Abre un issue** en el repositorio de GitHub
- **Contacta directamente** al equipo de desarrollo

CONCLUSIONES Y RECOMENDACIONES

El desarrollo de GestIAPro demuestra la viabilidad de integrar tecnologías modernas como inteligencia artificial, comunicación en tiempo real y arquitectura MERN en una solución robusta para la gestión colaborativa de proyectos. A través de una estructura modular, una interfaz intuitiva y funcionalidades orientadas al usuario como la generación automática de historias de usuario y el chat multilingüe, el sistema mejora significativamente la coordinación y productividad de los equipos. A pesar de los desafíos técnicos enfrentados, el equipo logró implementar soluciones eficaces que fortalecen la seguridad, escalabilidad y mantenibilidad del sistema, quedando el entorno listo para su puesta en producción tras resolver errores menores en el despliegue.

Se recomienda concluir la estabilización del despliegue en Railway mediante una revisión detallada de configuraciones y dependencias, así como ampliar la cobertura de pruebas automatizadas para asegurar la calidad del software. Asimismo, es importante mejorar la documentación técnica, evaluar el rendimiento de los modelos de IA integrados, fomentar la participación equitativa de todos los miembros del equipo y considerar el uso de contenedores Docker para una mayor portabilidad. Finalmente, si se busca escalar la solución para entornos reales, será necesario realizar pruebas de carga, reforzar la seguridad y explorar integraciones con herramientas externas de desarrollo y gestión de proyectos.